Vote Runner — Master Build Prompt

How to use: copy everything below the line into your AI coding tool (Antigravity, Cursor, Claude Code, etc.). It builds the game phase by phase.

ROLE

You are a senior game developer and software architect. Build a complete, mobile-first endless-runner web game called **Vote Runner**, in the style of Subway Surfers and Temple Run. Work phase by phase (section 7). Do not ask questions; make sensible assumptions and list them at the end of each phase.

1. CONCEPT

A satirical meme game. The player controls a campaign leader, **Neta Ji**, who runs down an endless road collecting **votes**. A rival, **Vipaksh Bhai**, runs behind and tries to catch him. Every stumble lets the rival get closer. When he catches Neta Ji, the run ends.

Hard rule: fictional characters only

- Both characters are fictional cartoon caricatures. Do not use any real politician's name, face, voice or quote, and no real party's name, symbol, logo, branding colors or slogan.
- Humor comes from situations (traffic jams, rallies, press mics, stray cows), never from real people, religion, caste or community.
- Both characters are treated equally. Neither is the hero or the villain beyond the chase mechanic.
- Names, art paths and speech lines live only in `src/config/characters.ts`, never hardcoded elsewhere.

2. TECH STACK

- TypeScript (strict mode) + Vite
- Three.js: 3D perspective camera, low-poly road and props, characters as 2D cartoon sprites used as billboards
- Web Audio API through one small `AudioManager` (placeholder beeps generated in code if no files exist)

- `localStorage` for saves, always wrapped in try/catch
- PWA (manifest + service worker), portrait mobile-first, desktop keyboard also supported
- Static build, deployable on Cloudflare Pages. Optional later: Capacitor wrapper for Android
- No backend in v1
- Vitest for pure-logic tests (spawner validator, scoring, chase gap)
- The game must run with zero external art: draw placeholder characters and props with primitives or canvas, and let real art drop in later through config paths

3. GAME STRUCTURES

3.1 Core types

```
export type LaneIndex = -1 | 0 | 1;
export const LANE_WIDTH = 2.2;

export type PlayerAction = 'running' | 'jumping' | 'sliding' | 'stumbling' |
'dead';
export type PowerUpId = 'magnet' | 'multiplier' | 'shield' | 'helicopter' |
'slowmo';

export interface PlayerState {
  lane: LaneIndex;
  x: number;                // smoothed horizontal position
  y: number;                // height above ground
  vy: number;               // vertical velocity
  action: PlayerAction;
  invulnerableFor: number;  // seconds left after a stumble
  shieldCharges: number;
  activePowerUps: Partial<Record<PowerUpId, number>>; // seconds left
}

export interface ChaserState {
  gap: number;              // distance behind the player, 0..GAP_MAX
  mode: 'chasing' | 'closing' | 'caught';
}

export type ObstacleKind = 'low' | 'high' | 'full' | 'moving';
```

```
// low = jump over, high = slide under, full = change lane, moving = crosses
lanes

export interface ObstacleDef {
  id: string;
  kind: ObstacleKind;
  size: { w: number; h: number; d: number };
  lanesBlocked: 1 | 2 | 3;
  art: string;
  minDifficulty: number;     // 0..1, when it starts appearing
}

export interface PowerUpDef {
  id: PowerUpId;
  name: string;
  duration: number;          // seconds
  art: string;
}

export type Cell =
  | { t: 'empty' }
  | { t: 'vote'; value: 1 | 5 }
  | { t: 'obstacle'; id: string }
  | { t: 'powerup'; id: PowerUpId };

export interface Row { z: number; cells: [Cell, Cell, Cell]; } // left,
middle, right

export interface Chunk {
  id: string;
  length: number;            // world units
  difficulty: number;        // 0..1
  rows: Row[];
}

export interface SaveData {
  version: 1;
  highScore: number;
  bestVotes: number;
  totalVotes: number;        // banked votes, spent on revive and skins
  settings: { sfx: boolean; music: boolean; haptics: boolean; reducedMotion:
boolean };
```

```
  unlockedSkins: string[];
  selectedSkin: string;
  missions: { id: string; progress: number; goal: number; done: boolean }[];
}
```

3.2 Constants (all in `src/config/constants.ts`)

Constant	Value
START_SPEED	12 units/s
MAX_SPEED	30 units/s
SPEED_RAMP	linear from start to max over 180 s
GRAVITY	-38
JUMP_VELOCITY	13
SLIDE_DURATION	0.7 s
LANE_SWITCH_TIME	0.12 s
ROW_SPACING	3 units
SPAWN_HORIZON	90 units ahead of the player
GAP_START / GAP_MAX	10
STUMBLE_GAP_LOSS	6
GAP_RECOVERY	0.4 per second while running clean
STUMBLE_INVULN	1.2 s
INPUT_BUFFER	150 ms
MIN_LANE_CHANGE_INTERVAL	0.25 s (used by the spawner validator)

3.3 Content lists

Obstacles

id	kind	How to beat it
barricade	low	jump
pothole	low	jump
press_mic	high	slide (mic boom hangs overhead)
rally_banner	high	slide
traffic_car	full	change lane
stray_cow	moving	walks across lanes slowly; time the lane change

Every obstacle needs a shape and color cue so it is readable without color (colorblind-safe).

Collectibles

- Vote (ballot-slip token): value 1
- Golden vote: value 5

Power-ups (8 s unless stated)

id	Name	Effect
magnet	Campaign Jeep	pulls votes from all 3 lanes
multiplier	Viral Reel	score x2
shield	Security Cover	absorbs one stumble (lasts 10 s)
helicopter	Helicopter	fly above all obstacles for 6 s, rows of votes in the air
slowmo	Chai Break	world speed x0.7

3.4 Chunk authoring format

Chunks are written as ASCII in `src/config/chunks.ts` and parsed into `Chunk` objects. One line = one row = 3 characters (left, middle, right).

```
legend: . empty   v vote   V golden vote
        B barricade   P pothole   M press_mic   R rally_banner   C
traffic_car   @ stray_cow
        m magnet   x multiplier   s shield   h helicopter   z slowmo

chunk 'zigzag' (difficulty 0.1)
v..
.v.
..v
.v.
v..

chunk 'jump-and-shift' (difficulty 0.3)
.v.
.B.
.v.
C.v
v..
```

Author at least 25 chunks across 4 difficulty tiers (0-0.25, 0.25-0.5, 0.5-0.75, 0.75-1).

3.5 Rules

- **Stumble:** if the player hits an obstacle while not invulnerable and without a shield: `gap -= STUMBLE_GAP_LOSS`, action becomes `stumbling` for 0.4 s, `invulnerableFor = STUMBLE_INVULN`. With a shield: remove one charge, no gap loss.
- **Caught:** when `gap <= 0`, the run ends. Two stumbles within about 5 seconds always end the run.
- **Recovery:** while running with no stumble, `gap` regrows at GAP_RECOVERY up to GAP_MAX.
- **Score:** `score = floor(distance) + votesCollected * 10 * multiplier`. Show Votes and Distance separately in the HUD.
- **Difficulty (0..1):** `min(1, distance / 3000)`. It controls speed, which chunk tiers are allowed and which obstacles unlock.

3.6 State machine

```
Boot -> Menu -> Playing <-> Paused
Playing -> GameOver
GameOver -> Revive offer (once per run, costs 50 banked votes: gap set to 6,
2 s invulnerable) -> Playing
GameOver -> Menu | Playing (new run)
```

4. ARCHITECTURE

4.1 Folder structure

```
vote-runner/
  index.html
  vite.config.ts
  public/            manifest.webmanifest, sw.js, icons/, audio/, sprites/
  src/
    main.ts
    config/          constants.ts, characters.ts, obstacles.ts, powerups.ts,
                     chunks.ts, difficulty.ts, themes.ts, lines.ts
    core/            Game.ts, StateMachine.ts, EventBus.ts, Pool.ts, Time.ts
    world/           WorldManager.ts, Track.ts, ChunkSpawner.ts,
                     SpawnValidator.ts, Environment.ts, CameraRig.ts
    entities/        Player.ts, Chaser.ts, Obstacle.ts, Collectible.ts,
PowerUp.ts
    systems/         InputSystem.ts, MovementSystem.ts, CollisionSystem.ts,
                     ChaseSystem.ts, PowerUpSystem.ts, ScoreSystem.ts,
MissionSystem.ts
    ui/              HUD.ts, MenuScreen.ts, PauseScreen.ts,
GameOverScreen.ts,
                     SettingsScreen.ts, ShopScreen.ts   (DOM overlay on top
of the canvas)
    audio/           AudioManager.ts
    storage/         SaveManager.ts
    utils/           math.ts, rng.ts
  tests/
  README.md
```

4.2 Core design rules

1. **Fixed timestep.** Update at 1/60 s with an accumulator, render with interpolation, clamp large delta times (tab switches).
2. **The player stays at z = 0 and the world scrolls toward the camera.** This avoids floating-point drift on long runs.
3. **Object pooling** for obstacles, collectibles, power-ups and road segments. No allocations inside the update loop.
4. **Systems communicate through a typed `EventBus`.** Events: `vote:collected`, `stumble`, `powerup:start`, `powerup:end`, `caught`, `score:changed`, `state:changed`, `mission:progress`.
5. **Data-driven content.** Obstacles, power-ups, chunks, themes and text lines are config only. Game logic never hardcodes content.
6. **HUD and menus are DOM elements** over the canvas, for crisp text. Use a font stack that renders both English and Hindi.
7. **Performance budget:** 60 FPS on a mid-range Android phone, at most about 150 draw calls, pixel ratio capped at 2, fog to hide pop-in, and a low-quality mode that lowers pixel ratio automatically if FPS drops under 45.

4.3 Main loop (pseudocode)

```
function frame(now):
  acc += clamp(now - last, 0, 0.1)
  while acc >= STEP:
    if state == Playing:
      input.consume()          // buffered swipes and keys
      movement.update(STEP)    // lane lerp, jump, slide, gravity
      world.update(STEP)       // scroll, recycle segments, spawn chunks
      collision.update()       // only entities in the nearest 2 rows
      powerups.update(STEP)
      chase.update(STEP)       // gap recovery, caught check
      score.update(STEP)
    acc -= STEP
  render(alpha = acc / STEP)
```

4.4 System details

InputSystem

- Touch swipe (threshold 30 px): left/right = lane, up = jump, down = slide. Keyboard: arrows and WASD. Pause button and Esc.
- Queue one input for INPUT_BUFFER ms so inputs just before landing still count.

MovementSystem

- Lane change by smooth lerp over LANE_SWITCH_TIME. Jump uses JUMP_VELOCITY and GRAVITY. Slide shrinks the hitbox for SLIDE_DURATION. Pressing down in the air fast-falls.

ChunkSpawner and SpawnValidator

- Keep chunks spawned up to SPAWN_HORIZON ahead. Pick the next chunk by weighted random from the tiers allowed at current difficulty. Use a seeded RNG so runs can be replayed in tests.
- Validator rules: every row has at least one passable lane; the route is reachable with the actions needed; no two lane changes are required closer than MIN_LANE_CHANGE_INTERVAL at the current speed. Reject and re-pick any chunk that fails.

CollisionSystem

- AABB in lane space (x, y, z). Compare the player only with entities in the nearest 2 rows. Obstacles return `stumble`, collectibles return `collect`.

ChaseSystem

- Owns `gap`. The rival sprite is drawn behind the player at a distance mapped from `gap` (visible at the bottom of the screen when gap is under 5). Footstep audio and a screen-edge vignette get stronger as gap shrinks. Speech bubbles from `lines.ts` appear every 8 to 15 seconds.

PowerUpSystem

- A timer map on the player. HUD shows icons with circular countdowns. Picking up the same power-up refreshes its timer.

MissionSystem

- 3 daily missions picked from templates (collect N votes, jump N times, run N meters, use N power-ups). Progress is saved. Completing one grants banked votes.

AudioManager

- Channels: music, sfx. Unlock the audio context on first touch. Respect settings toggles. Sounds: vote, golden vote, jump, slide, stumble, power-up start/end, caught, UI click.

CameraRig

- Chase camera behind and above the player, slight lag on lane changes, small FOV increase with speed, short screen shake on stumble (disabled if reducedMotion).

5. CHARACTERS AND ART

- **Neta Ji:** cartoon character in a light kurta and a cap. Animations: run (6 frames), jump, slide, stumble, dead.
- **Vipaksh Bhai:** cartoon character in a different color kurta, the same animation set, a bit bulkier, with a megaphone.
- Both are generic caricatures with invented faces and no real-party colors or symbols.
- `characters.ts` holds id, display name, sprite path, frame counts and speech lines.
- Placeholder art: colored capsule bodies with simple faces drawn on a canvas texture.
- Skins (cosmetic only, phase 7): kurta colors, caps and vote-trail effects.

6. THEMES, TEXT AND JUICE

Themes by distance

Distance	Theme
0-500 m	Main Street (shops, rickshaws, banners)
500-1200 m	Busy Market
1200-2500 m	Flyover and Highway
2500 m+	Night Rally Ground

Transitions blend fog color and prop sets smoothly.

Text lines (neutral, in `lines.ts`)

- Rival: Ruko bhai, ruko! / Ek minute baat suno! / Itni jaldi kya hai? / Bas thoda aur!
- Pop-ups: +Vote! / Rally mode! / Breaking news! / Chai time!
- Game-over ticker: BREAKING: Neta Ji runs {distance} m and collects {votes} votes before being caught.

Juice

- Vote pickup: small pop, floating +1, counter bump
- Golden vote: bigger burst
- Milestone every 500 m with a banner
- Particle dust on land, haptics via `navigator.vibrate` on stumble if enabled

7. BUILD PHASES (do in order, test each)

Phase	Build	Done when
0	Vite + TS + Three.js setup, folder structure, EventBus, StateMachine, fixed-timestep loop	Empty scene renders, FPS counter shows 60
1	Endless road with recycled segments, player with lane switch, jump, slide, chase camera, all inputs	Smooth control on phone and keyboard
2	Config content, chunk parser and spawner, validator, obstacles, votes, collision, stumble	A 2-minute run plays with no impossible rows
3	Chaser and gap logic, caught state, GameOver screen with ticker	Game is fully playable start to end
4	Power-ups, score, HUD, difficulty ramp	All 5 power-ups work and speed ramps up
5	Menu, pause, settings, SaveManager, missions, revive	Progress persists after reload
6	Audio, juice, themes, PWA, performance pass, README, deploy build	60 FPS on a mid-range phone, installable PWA
7 (optional)	Skins and shop, leaderboard, Capacitor Android build	

8. QUALITY RULES AND OUTPUT FORMAT

- After each phase: run the build and tests, then report (a) what now works, (b) how to test it, (c) assumptions made, (d) the next phase. Then continue.

- Keep files small and typed, no `any`.
- Handle edge cases: tab hidden pauses the game, window resize and orientation change, corrupted save data resets safely, input ignored while paused, WebGL context loss.
- Accessibility: reduced-motion toggle, sound toggles, large tap targets (at least 44 px), shape-based cues for obstacles.
- Write unit tests for: SpawnValidator, ScoreSystem, ChaseSystem gap math, SaveManager migration.
- README must explain how to run, build, deploy, and how to swap character names and art in `characters.ts`.
- Do not add real political names, faces, parties or symbols anywhere, even as placeholders or comments.

Start now with Phase 0 and Phase 1.